React Concurrent Features Cheatsheet

Modern React patterns for non-blocking updates and better UX.

Suspense

Declarative loading boundaries for code splitting and async data sources.

Basic API

```
<Suspense fallback={<LoadingSkeleton />}>
  <Component />
</Suspense>
```

Lazy loading

Loads components on-demand with automatic fallback UI.

```
const LazyComponent = React.lazy(() => import('./
Heavy'));

<Suspense fallback={<Skeleton />}>
  <LazyComponent />
</Suspense>
```

Async data sources

Components suspend and show Suspense fallback while async data loads.

```
const userPromise = fetchUser(`/api/user/${userId}`);

function UserProfile({ userPromise }) {
  const user = use(userPromise); // Suspends until
resolved
  return <div>{user.name}</div>;
}

async function UserProfile({ userId }) {
  const user = await fetchUser(`/api/user/${userId}`);
  return <div>{user.name}</div>;
}
```

- Declarative: No manual loading states or conditional rendering
- Universal: Works with `React.lazy()`, `use()` API, async Server Components, and data libraries
- Composable: Nest multiple boundaries for granular loading control

useTransition

Mark updates as non-urgent, creating Actions. Useful for expensive rendering, async operations, and opting out of Suspense.

Basic API

```
const [isPending, startTransition] = useTransition();
```

Expensive rendering

Prevents blocking during heavy computations while showing pending state.

```
<button
  onClick={() => startTransition(() =>
setActiveTab('profile'))}
  style={{ opacity: isPending ? 0.7 : 1 }}
>
  Profile
</button>
```

Async operations

Coordinates state updates with server requests, while automatically providing a built-in loading state.

```
<button
  onClick={() => startTransition(async () => {
    const error = await deletePost(postId);
    startTransition(() => {
      setError(error);
    });
  })}
  disabled={isPending}
>
  {isPending ? 'Deleting...' : 'Delete'}
</button>
```

Opting out of Suspense

Keeps current content visible instead of showing fallback during updates.

```
const [currentPage, setCurrentPage] =
useState('home');

function navigateTo(page) {
  startTransition(() => setCurrentPage(page));
}
```

- Non-blocking: Keep UI responsive during expensive updates
- Built-in pending: Get loading state without extra `useState`
- Auto-batching: Multiple state updates batched automatically, preventing intermediate renders that cause UI flicker

React Concurrent Features Cheatsheet

Modern React patterns for non-blocking updates and better UX.

Suspense

Declarative loading boundaries for code splitting and async data sources.

Basic API

```
<Suspense fallback={<LoadingSkeleton />}>
  <Component />
</Suspense>
```

Lazy loading

Loads components on-demand with automatic fallback UI.

```
const LazyComponent = React.lazy(() => import('./
Heavy'));

<Suspense fallback={<Skeleton />}>
  <LazyComponent />
</Suspense>
```

Async data sources

Components suspend and show Suspense fallback while async data loads.

```
const userPromise = fetchUser(`/api/user/${userId}`);

function UserProfile({ userPromise }) {
  const user = use(userPromise);

  return <div>{user.name}</div>;
}

async function UserProfile({ userId }) {
  const user = await fetchUser(`/api/user/${userId}`);
  return <div>{user.name}</div>;
}
```

- Declarative: No manual loading states or conditional rendering
- Universal: Works with `React.lazy()`, `use()` API, async Server Components, and data libraries
- Composable: Nest multiple boundaries for granular loading control

useTransition

Mark updates as non-urgent, creating Actions. Useful for expensive rendering, async operations, and opting out of Suspense.

Basic API

```
const [isPending, startTransition] = useTransition();
```

Expensive rendering

Prevents blocking during heavy computations while showing pending state.

```
<button
  onClick={() => startTransition(() =>
setActiveTab('profile'))}
  style={{ opacity: isPending ? 0.7 : 1 }}
>
  Profile
</button>
```

Async operations

Coordinates state updates with server requests, while automatically providing a built-in loading state.

```
<button
  onClick={() => startTransition(async () => {
    const error = await deletePost(postId);
    startTransition(() => {
      setError(error);
    });
  })}
  disabled={isPending}
>
  {isPending ? 'Deleting...' : 'Delete'}
</button>
```

Opting out of Suspense

Keeps current content visible instead of showing fallback during updates.

```
const [currentPage, setCurrentPage] =
useState('home');

function navigateTo(page) {
  startTransition(() => setCurrentPage(page));
}
```

- Non-blocking: Keep UI responsive during expensive updates
- Built-in pending: Get loading state without extra `useState`
- Auto-batching: Multiple state updates batched automatically, preventing intermediate renders that cause UI flicker